

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Створення інтегрованих середовищ розробки»

Робота №1

Тема «Лексичний аналізатор»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Льовкін В.М.

2025

МЕТА

Метою роботи є ознайомитись з основами процесу виконання лексичного аналізу вихідної програми та інструментами, які використовуються під час реалізації даного процесу; навчитися проєктувати та розробляти лексичний і синтаксичний аналізатори у складі компілятора за допомогою генератора парсерів ANTLR.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.
2. Визначити граматику для виконання лексичного аналізу програм мовою, заданою у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем.

Розробити IDE для мови програмування Rust. Обов'язково мають бути проаналізовані наступні можливості даної мови: вбудовані типи даних, функції, вектори, записи, методи, цикли, умовні оператори, зв'язування змінних, арифметичні оператори.

3. Побудувати кінцеві автомати для представлення сформованої граматики.

4. Побудувати лексичний аналізатор за допомогою генератора ANTLR мовою програмування Java:

- програма повинна виконувати лексичний аналіз вхідного тексту програми у відповідності з визначеним індивідуальним завданням;
- програма повинна обробляти і виводити повідомлення про всі помилки, виявлені під час лексичного аналізу з зазначенням причини помилки (недопустимі символи, пропущені лапки, тощо);

- програма повинна мати графічний інтерфейс та дозволяти задавати вхідний код програми для аналізу, задаючи його безпосередньо в інтерфейсі або створюючи файл програми, завантажуючи його через інтерфейс та виводячи результати лексичного аналізу.

5. Створити git-репозиторій для роботи над проєктом та додати в нього розроблений лексичний аналізатор.

6. Виконати тестування побудованого лексичного аналізатора.

7. Оформити звіт.

8. Відповісти на контрольні питання.

ВИКОНАННЯ

1 Лексична граматика

```
grammar Rust;

// --- PARSER ---
start: block | method | function | struct | if | else | if_else | else_if;

// Block
block: LEFT_CURLY_BRACE .*? RIGHT_CURLY_BRACE;
method: FN ID LEFT_PARENTHESIS .*? RIGHT_PARENTHESIS RIGHT_ARROW? ID? block;
function: FN ID LEFT_PARENTHESIS .*? RIGHT_PARENTHESIS block;
struct: STRUCT ID LEFT_CURLY_BRACE .*? RIGHT_CURLY_BRACE | IMPL ID block;
if: IF .*? block;
else: ELSE .*? block;
if_else: if else;
else_if: ELSE IF .*? block;

// Inline
variable: LET MUT? ID COLON? (data_types)? EQ binding;
binding: FLOAT_LIT | INT_LIT | STRING_LIT | BOOL_LIT;
vector: VEC_MACRO LEFT_SQUARE_BRACE .*? RIGHT_SQUARE_BRACE;
loop: LOOP block | WHILE .*? block | FOR .*? block;

// Helper
data_types: INTEGER | UNSIGNED_INTEGER | FLOAT | BOOL | CHAR | STRING;

// --- LEXER ---
// Space and commentaries
WS: [ \t\r\n]+ -> skip;
SINGLE_COMMENT: '//' ~[\r\n]* -> skip;
DOUBLE_COMMENT: '/*' .*? '*/' -> skip;

// Key words
```

```

AS: 'as';
ASYNC: 'async';
AWAIT: 'await';
BREAK: 'break';
CONST: 'const';
CONTINUE: 'continue';
CRATE: 'crate';
DYN: 'dyn';
ELSE: 'else';
ENUM: 'enum';
EXTERN: 'extern';
FALSE: 'false';
FN: 'fn';
FOR: 'for';
IF: 'if';
IMPL: 'impl';
IN: 'in';
LET: 'let';
LOOP: 'loop';
MATCH: 'match';
MOD: 'mod';
MOVE: 'move';
MUT: 'mut';
PUB: 'pub';
REF: 'ref';
RETURN: 'return';
SELF: 'self';
STATIC: 'static';
STRUCT: 'struct';
SUPER: 'super';
TRAIT: 'trait';
TRUE: 'true';
TYPE: 'type';
USE: 'use';
WHERE: 'where';
WHILE: 'while';

// Literals and identifiers
BOOL_LIT: 'true' | 'false';
INT_LIT: [0-9]+;
FLOAT_LIT: [0-9]+ '.' [0-9]+ | '.' [0-9]+;
STRING_LIT: '"' (~["\\] | '\\' .)* '"';
ID: [a-zA-Z_] [a-zA-Z0-9_]*;

// Data types
INTEGER: 'i8' | 'i16' | 'i32' | 'i64' | 'i128' | 'isize';
UNSIGNED_INTEGER: 'u8' | 'u16' | 'u32' | 'u64' | 'u128' | 'usize';
FLOAT: 'f32' | 'f64';
BOOL: 'bool';
CHAR: 'char';
STRING: 'str' | 'String';

// Brackets
LEFT_CURLY_BRACE: '{';
RIGHT_CURLY_BRACE: '}';
LEFT_SQUARE_BRACE: '[';

```

```

RIGHT_SQUARE_BRACE: ']' ;
LEFT_PARENTHESIS: '(' ;
RIGHT_PARENTHESIS: ')' ;

// Symbols
EXCLAMATION: '!' ;
EXCLAMATORY_EQUAL: '!=' ;
PERCENT: '%' ;
AMPERSAND: '&' ;
AMPERSAND_EQUAL: '&=' ;
AMPERSAND_AMPERSAND: '&&' ;
STAR: '*' ;
STAR_EQUAL: '*=' ;
PLUS: '+' ;
PLUS_EQUAL: '+=' ;
COMMA: ',' ;
DASH: '-' ;
DASH_EQUAL: '-=' ;
RIGHT_ARROW: '->' ;
DOT: '.' ;
DOT_DOT: '..' ;
DOT_DOT_EQUAL: '..=' ;
DOT_DOT_DOT: '...' ;
SLASH: '/' ;
SLASH_EQUAL: '/=' ;
COLON: ':' ;
SEMICOLON: ';' ;
LEFT_LEFT: '<<' ;
LEFT_LEFT_EQUAL: '<<=' ;
LEFT: '<' ;
LEFT_EQUAL: '<=' ;
EQUAL: '=' ;
EQUAL_EQUAL: '==' ;
RIGHT_ARROW_BIG: '=>' ;
RIGHT: '>' ;
RIGHT_EQUAL: '>=' ;
RIGHT_RIGHT: '>>' ;
RIGHT_RIGHT_EQUAL: '>>=' ;
AT: '@' ;
CARET: '^' ;
CARET_EQUAL: '^=' ;
PIPE: '|' ;
PIPE_EQUAL: '|=' ;
PIPE_PIPE: '||' ;
QUESTION: '?' ;
UNDERSCORE: '_' ;
PATH_SEPARATOR: '::' ;
POUND: '#' ;
DOLLAR: '$' ;

// Vector macro
VEC_MACRO: 'vec!' ;

```

2 Кінцеві автомати, які використовуються під час аналізу

Кінцеві автомати наведено на рисунках нижче:

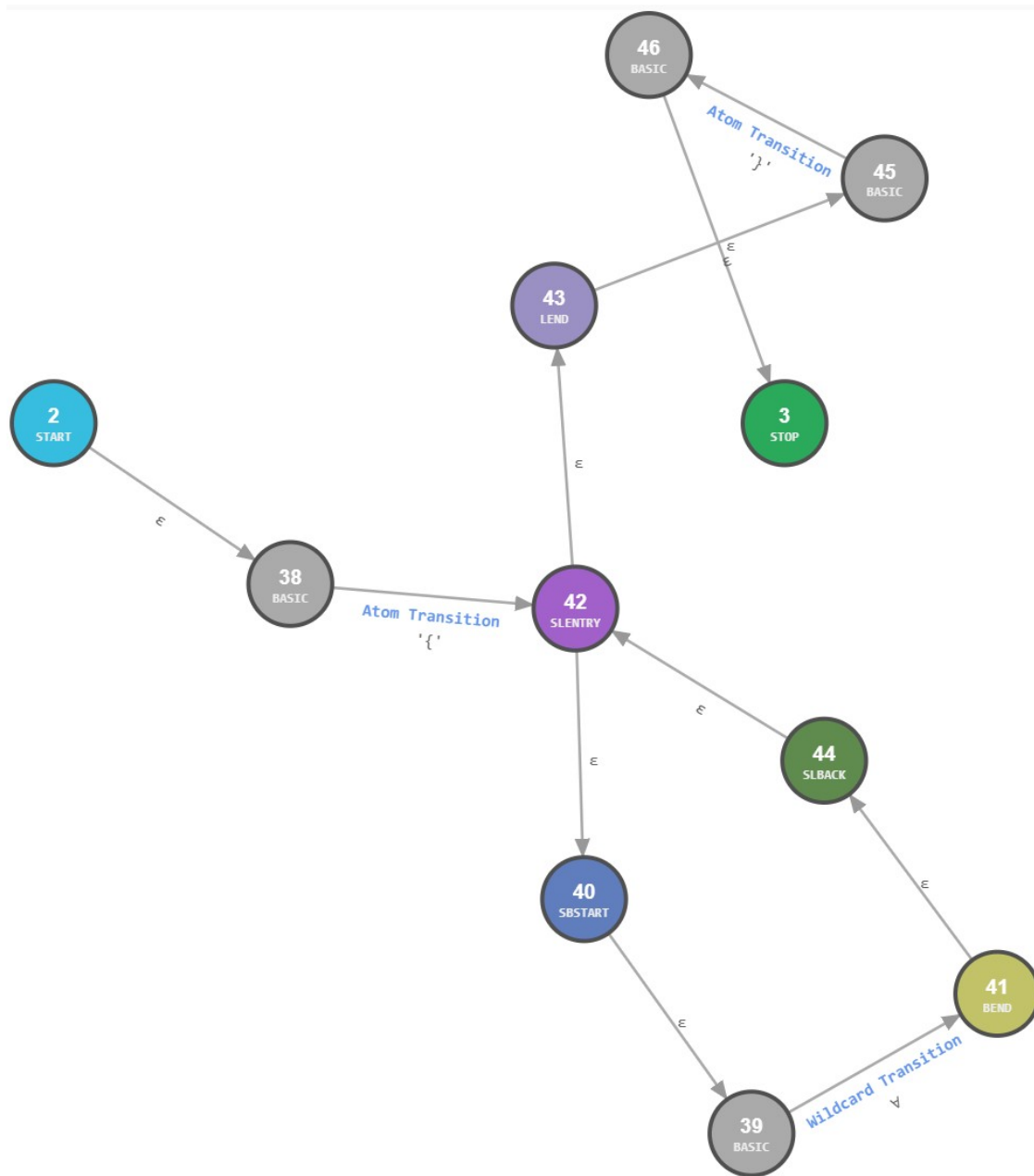


Рисунок 2.1 — Автомат *block*

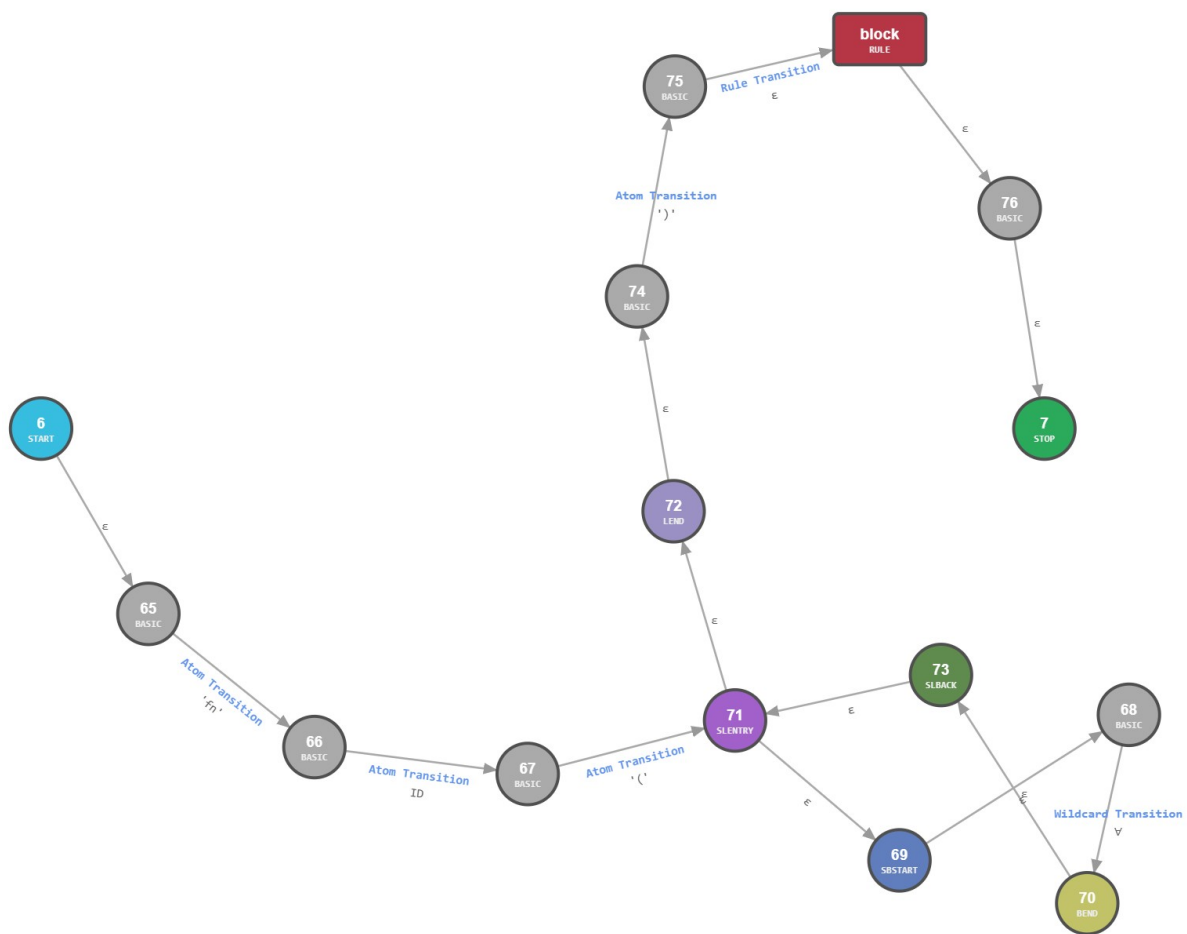


Рисунок 2.2 — Автомат *function*

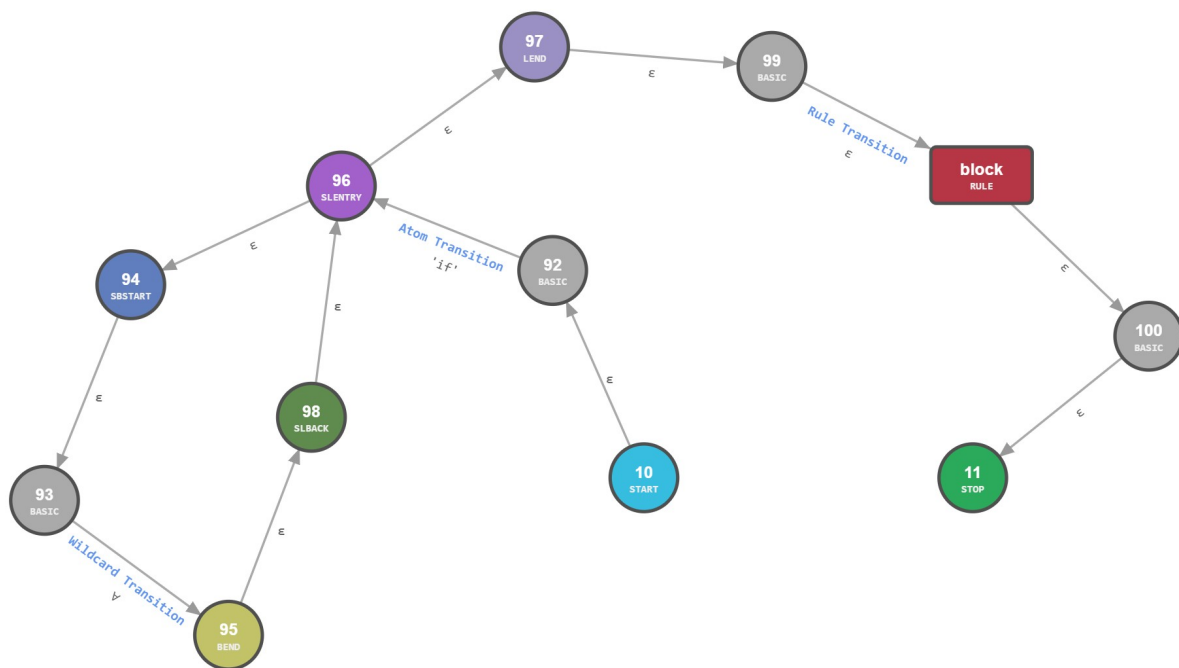


Рисунок 2.3 — Автомат *if*

3 Інтерфейс роботи з програмою в декількох режимах

Вигляд програми наведено нижче на рисунках:

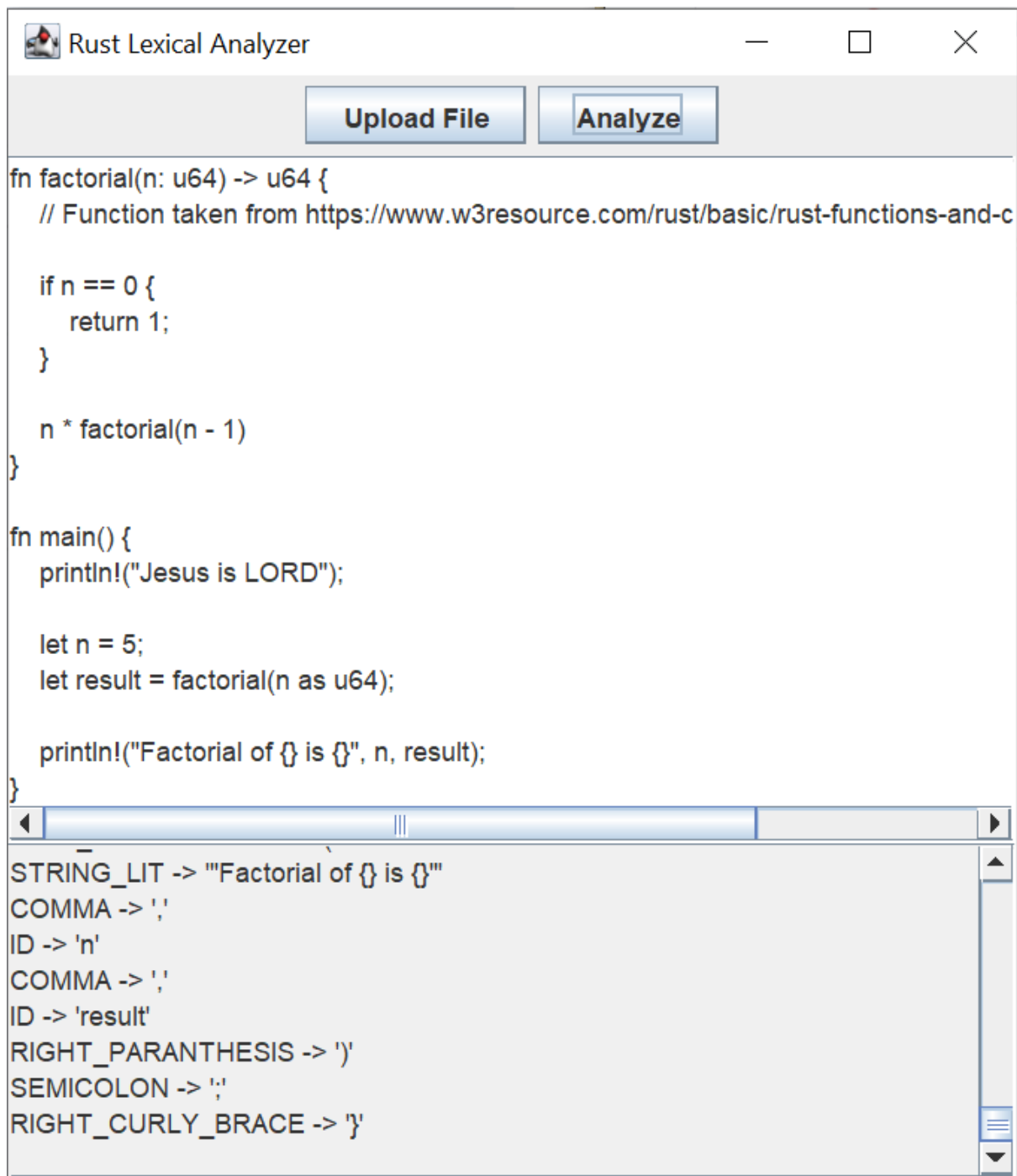


Рисунок 3.1 — Інтерфейс програми

ВИСНОВКИ

Благодаттю Господа Ісуса Христа зроблено роботу. Розроблено лексичний аналізатор мовою Antlr та програму мовою Java.

ПИТАННЯ

З чого складається сучасне інтегроване середовище розробки?

Складається з багатьох частин, зокрема:

- текстовий редактор;
- менеджер проєктів;
- інтеграція з системами контролю версій.

Чим відрізняються компілятори та інтерпретатори?

Компілятор перекладає весь код у машинний одразу, а інтерпретатор виконує його рядок за рядком.

Що є результатом та входом лексичного аналізатору?

Результатом лексичного аналізатору є потік токенів.